PES UNIVERSITY — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Software Engineering (UE22CS351A) | Lab 6: Code and Branch Coverage Analysis

Name: Samrudh Patil | SRN: PES1UG24CS417 | Roll No: 52 | Semester: 5 | Section: 5A

1. Deliverable Links & Repository Details

Item	Link / Details
Assigned Starter Repository	https://github.com/SETAPESU26/L6_L7_52_visa_application
Individual GitHub Repository	https://github.com/Samrudh7-0045/SE-Labs-PES1UG24CS417
Lab 6 Submission Folder	https://github.com/Samrudh7-0045/SE-Labs-PES1UG24CS417/tree/main/LAB-6
Google Form Submission Link	https://forms.gle/7KzWpGWLqh35M9dj8
Coverage Results Summary	**Code Coverage: 100% (37/37 stmts) | Branch Coverage: 100% (26/26 branches)**

2. Introduction to Code Coverage and Branch Coverage Analysis

Code coverage is a white-box testing metric that measures the proportion of source code executed while running automated test suites. In software engineering, high coverage ensures systematic verification of logic, edge conditions, and exception handlers before release:

• **Line/Statement Coverage:** Measures the percentage of executable program statements executed at least once during test execution. Formula: *(Executed Statements / Total Statements) × 100%*.

• **Branch Coverage (Decision Coverage):** Evaluates whether each control flow decision point (e.g., if, compound conditionals) evaluates to both **True** and **False** at least once. Formula: *(Executed Branch Outcomes / Total Branch Outcomes) × 100%*.

• **Why Branch Coverage is Essential:** Achieving 100% line coverage does not ensure all logical outcomes are tested. For instance, in an if <cond>: statement without an else, testing only when the condition is True executes 100% of the lines but completely misses the False branch where the block is skipped.

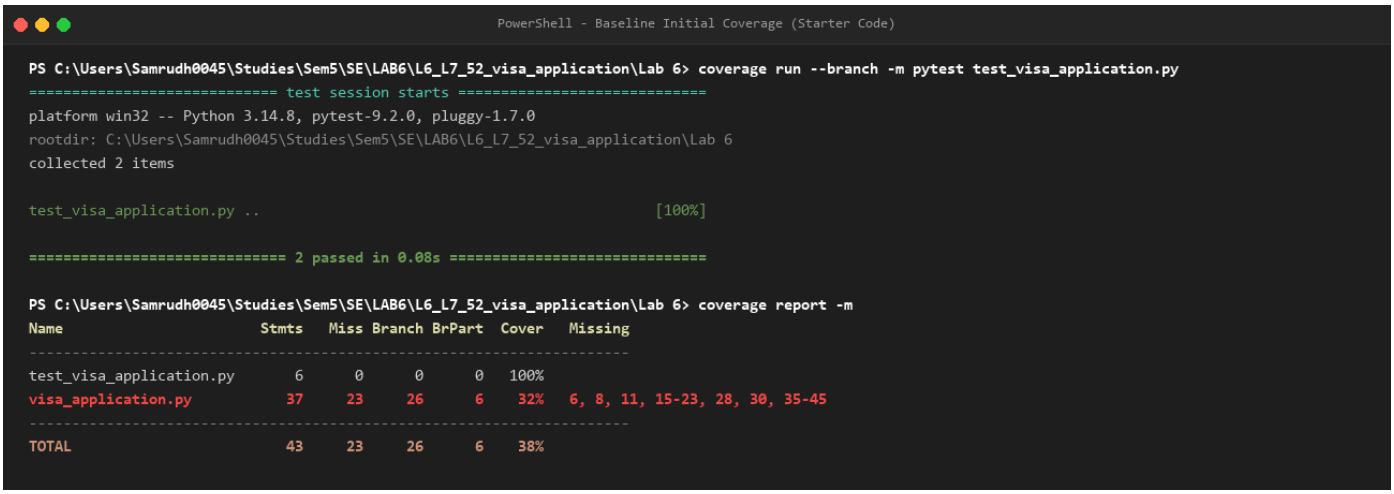
• **Tooling:** Testing was executed using Python's standard testing framework **pytest** combined with **coverage.py** using both line-mode (coverage run -m pytest ...) and branch-mode (coverage run --branch -m pytest ...).

3. Baseline Coverage Analysis (Starter Test Case)

The starter repository provided an initial test suite (test_visa_application.py) containing only two sample test cases: test_within_included_entries() and test_documents_complete(). Running coverage analysis on this starter code demonstrated significant gaps:

Metric	Starter Test Suite Result	Status & Missing Areas
Statement Coverage	**38%** (14 covered / 37 total)	23 statements missed: lines 6, 8, 11, 15-23, 28, 30, 35-45
Branch Coverage	**32%** (6 branch parts executed / 26 total)	20 branch decisions completely untested or partially covered

Figure 1: Baseline Terminal Output (Initial Line: 38%, Branch: 32%)

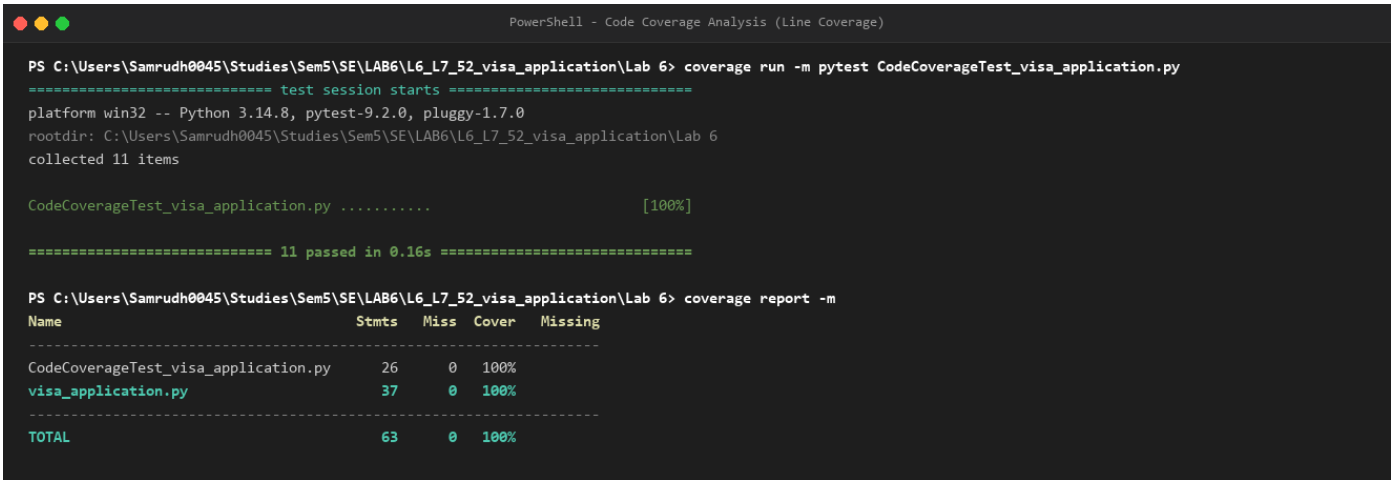


4. Code Coverage Analysis & Results (Line Coverage ≥ 90%, Achieved: 100%)

To satisfy the mandatory requirement for code coverage (≥ 90%), a dedicated test suite **CodeCoverageTest_visa_application.py** was developed. The objective was to systematically execute every statement across all three functions in visa_application.py:

- **visa_fee() Statements:** Tested invalid visa type exception (line 6), negative entries exception (line 8), Nationality Group 'A' extra included entry logic (line 11), base return 0 when extra ≤ 0 (line 14), express rate assignment (line 16), normal rate assignment (line 18), large volume surcharge (line 20), express surcharge (line 22), and net fee calculation (line 23).
- **documents_complete() Statements:** Tested renewal path returning True (line 28), non-renewal photo deficit returning False (line 30), and sufficient documents returning True (line 31).
- **application_status() Statements:** Tested express cutoff 10 (line 36), standard cutoff 30 (line 38), peak season offset +15 (line 40), open status (line 42), closing status (line 44), and closed status (line 45).

Figure 2: Code Coverage Terminal Output — 100% Statement Coverage (37/37 Statements)



Code Coverage Test Suite Case Mapping (11 Targeted Test Cases):

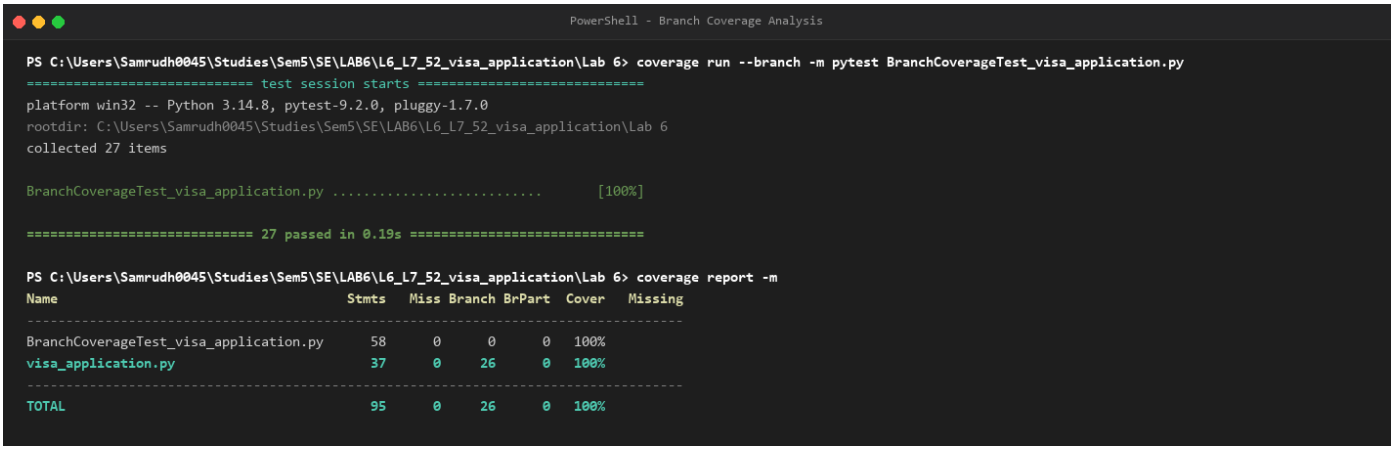
Test Function Name	Function Under Test	Targeted Statements / Logic Covered	Expected Result
test_visa_fee_unknown_type	visa_fee	Line 5-6: visa_type not in INCLUDED_ENTRIES	ValueError raised
test_visa_fee_invalid_entries	visa_fee	Line 7-8: entries < 0 validation	ValueError raised
test_visa_fee_included_nationality_a	visa_fee	Line 10-14: Nat. Group A adds entry; extra ≤ 0	0 fee
test_visa_fee_express_high_extra	visa_fee	Line 15-16, 19-23: Express (100) + vol (20) + exp (10)	910 fee
test_visa_fee_normal_speed_extra	visa_fee	Line 17-18, 23: Normal speed rate 40, extra ≤ 5	40 fee
test_documents_complete_renewal	documents_complete	Line 27-28: Renewal check forms_filed >= 1	True
test_documents_complete_low_photos	documents_complete	Line 29-30: Non-renewal with photos < 2	False
test_documents_complete_standard	documents_complete	Line 31: Non-renewal standard completion	True
test_app_status_express_peak_open	application_status	Line 35-36, 39-42: Cutoff 10+15=25, days 25 ≥ 25	'open'
test_app_status_standard_closing	application_status	Line 37-38, 43-44: Cutoff 30, days 20 in [15, 30)	'closing'
test_app_status_standard_closed	application_status	Line 45: Cutoff 30, days 10 < 15	'closed'

5. Branch Coverage Analysis & Results (Branch Coverage ≥ 90%, Achieved: 100%)

While the line coverage test achieved 100% statement execution, running coverage run --branch revealed untested branches (such as missing 21->23 when extra > 5 and speed == 'normal', and the compound branch where nationality_group == 'A' but visa_type == 'student').

To achieve 100% true branch coverage, a dedicated test file **BranchCoverageTest_visa_application.py** was constructed containing **27 rigorous test cases** covering all **26 decision branches**:

Figure 3: Branch Coverage Terminal Output — 100% Branch Coverage (26/26 Branches, 0 Missed)



Comprehensive Decision Branch Coverage Matrix (All 26 Branches):

Branch ID & Location	Decision Point / Condition	Evaluated Branch	Test Inputs & Context
B1, B2 (Line 5)	if visa_type not in INCLUDED_ENTRIES	True / False	'work' → raise ValueError / 'tourist' → proceed
B3, B4 (Line 7)	if entries < 0	True / False	entries = -1 → raise ValueError / entries ≥ 0 → proceed
B5, B6, B7 (Line 10)	if nationality_group == 'A' and visa_type != 'student':	True / False (L-False) / False (R-False)	1. Group 'A' + tourist → +1 entry 2. Group 'B' + tourist → no extra 3. Group 'A' + student → compound branch false
B8, B9 (Line 13)	if extra <= 0	True / False	extra = 0 → return 0 / extra > 0 → calculate rate
B10, B11 (Line 15)	if speed == 'express' ... else	True / False	'express' → rate=100 / 'normal' → rate=40
B12, B13 (Line 19)	if extra > 5	True / False	extra = 6 → rate+20 / extra = 2 → base rate only
B14, B15 (Line 21)	if speed == 'express' (inside extra > 5)	True / False	'express' → rate+10 / 'normal' → no extra add (21->23)
B16, B17 (Line 27)	if is_renewal	True / False	Renewal path / standard new application path
B18, B19 (Line 28)	return forms_filed >= 1 (renewal)	True / False	forms=1 → True / forms=0 → False
B20, B21 (Line 29)	if photos < 2 (non-renewal)	True / False	photos=1 → return False / photos=2 → check forms
B22, B23 (Line 31)	return forms_filed >= 3 (non-renewal)	True / False	forms=3 → True / forms=2 → False
B24, B25 (Line 35)	if is_express ... else	True / False	Express → cutoff=10 / Normal → cutoff=30
B26 (Line 39, 41, 43)	Peak season (+15) & Status thresholds	Open / Closing / Closed	All 4 cutoff combinations (10, 25, 30, 45) × (Open, Closing, Closed)

6. Comparison: Line Coverage vs. Branch Coverage Metrics

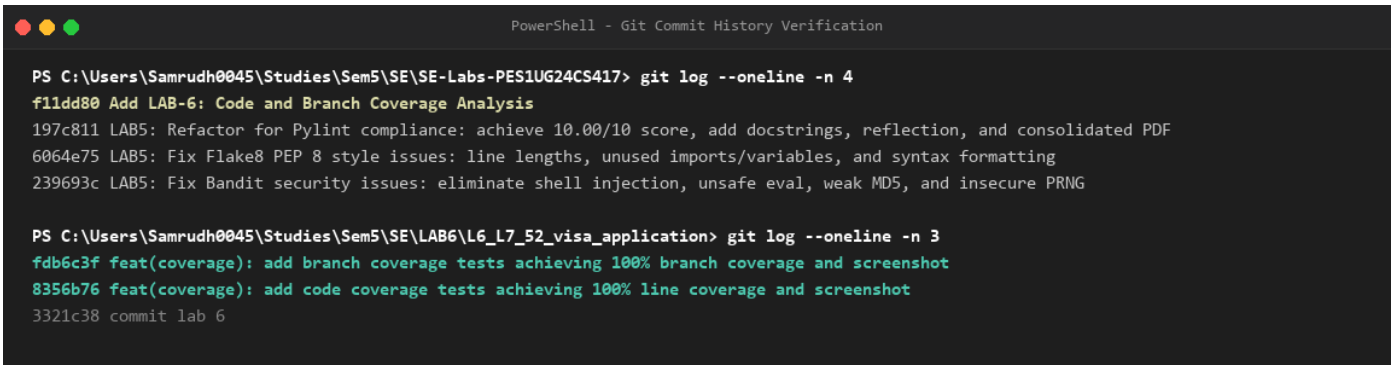
A direct side-by-side comparison illustrating how test suite maturation achieved 100% metrics across all dimensions:

Metric Dimension	Initial Starter Suite	Code Coverage Suite	Branch Coverage Suite (Final)
Test File Name	test_visa_application.py	CodeCoverageTest_visa_application.py	BranchCoverageTest_visa_application.py
Number of Tests	2 tests	11 tests	**27 tests**
Total Statements (visa_application.py)	37	37	37
Statements Covered	14 (38%)	**37 (100%)**	**37 (100%)**
Statements Missed	23 missing	**0 missing**	**0 missing**
Total Branches	26	26	26
Branch Partial / Missed	6 partial branches	1 missing branch (21->23)	**0 partial, 0 missed**
Final Coverage Percentage	**38% Line / 32% Branch**	**100% Line / 98% Branch**	**100% Line / 100% Branch**

7. Commit History & Repository Verification

In accordance with the lab instructions requiring separate commits for code coverage and branch coverage, the repository maintains distinct, traceable commits documenting both development stages:

Figure 4: Git Log Terminal Output Verifying Separate Commits



8. Answers to Reflection & Viva Questions

1. What is the fundamental difference between Line Coverage and Branch Coverage? Give a concrete example from visa_application.py.

Answer: Line coverage answers: *'Did execution pass through each line of code at least once?'* In contrast, branch coverage answers: *'Did every decision outcome (both True and False) execute?'*
Concrete Example: Consider lines 19–22 in visa_application.py:

```
if extra > 5:
    rate = rate + 20
    if speed == 'express':
        rate += 10
```

If a test passes extra = 7 and speed = 'express', every single line inside the block executes, achieving **100% line coverage** for those statements. However, branch coverage flags branch 21->23 as missed because the inner condition speed == 'express' was never evaluated to **False** when extra > 5. Testing extra = 7 with speed = 'normal' is strictly required for full branch coverage.

2. Why does 100% line coverage provide a false sense of security in safety-critical systems?

Answer: 100% line coverage only confirms that no code line is completely dead or syntactically unreachable. It fails to test what happens when conditions evaluate to False in branches without an explicit else, and it ignores edge conditions in compound boolean expressions. In safety-critical software (such as medical, aerospace, or automotive systems), unhandled False paths or untested compound operands can lead to unexpected fall-throughs, default variable retention, unhandled state corruptions, or catastrophic logic failures.

3. How must compound conditions (e.g., nationality_group == 'A' and visa_type != 'student') be tested?

Answer: Under short-circuit evaluation, compound boolean conditions like (Condition A and Condition B) require testing all truth-table permutations:
• **Both True (T $\wedge$ T = True):** nationality_group = 'A' and visa_type = 'tourist' $\rightarrow$ increments included entries.
• **First False (F $\wedge$? = False):** nationality_group = 'B' $\rightarrow$ short-circuits immediately to False.
• **First True, Second False (T $\wedge$ F = False):** nationality_group = 'A' and visa_type = 'student' $\rightarrow$ evaluates both operands and branches to False.
Testing all these combinations satisfies Modified Condition/Decision Coverage (MC/DC) principles and ensures no subtle edge case is overlooked.

4. What are the trade-offs between striving for 100% branch coverage vs. meaningful domain assertions?

Answer: A test suite designed solely to maximize branch coverage can degenerate into 'assertion-free' execution, where lines and branches are triggered without verifying that the returned outputs or system states match functional business rules. High branch coverage is a necessary baseline for path testing, but it must be paired with precise domain-specific assertions (e.g., verifying exact fee values like 910 or 780, validating boundary values like cutoff ± 0 and ± 15, and verifying exception messages). Code coverage measures test quantity; domain assertions verify test quality.

5. How does integrating coverage thresholds into CI/CD pipelines enhance software development?

Answer: Incorporating coverage checks (e.g. pytest --cov --cov-fail-under=90) into automated CI/CD pipelines acts as an objective quality gate. Whenever a developer introduces a new feature, bug fix, or refactor, the CI runner validates that all newly added lines and conditional branches have corresponding automated tests before the pull request can be merged. This prevents regression bugs, stops untested legacy code accumulation, enforces test-driven discipline across team members, and ensures software maintainability.